Sutra — Complete Project Overview

> A local-first, $0-by-default, voice-native personal AI that learns from experience, teaches you, writes its own skills, and acts on your real desktop — built solo by Sarjan. Codebase: `~/friday-os/` (internal package name `friday`, product name **Sutra**).

Compiled 2026-10-07 from the live codebase + full project memory.

---

0. In plain words — the thesis

Sutra is the **inversion** of the usual "AI assistant." Everyone else ships a chat box wrapping someone else's API and shipping your life to their servers. This is the opposite: **sovereign** (your memory graph and data never leave your machine), **$0 by default** (local Ollama → free tiers; paid Claude only when *you* flip the flag), and — the part most people don't attempt — it **learns from experience, teaches you back, and writes its own skills with your approval.** It's the assistant you'd build for yourself if you had the time. One engineer did, solo, in ~12 weeks of disciplined PR-per-feature work.

The architecture reflects a worldview, not just engineering. Tiered routing isn't a cost hack — it's sovereignty. The confirmation gate (enforced *and* CI-audited), `is_catastrophic`, the scope guard, and the "drafted skills are inert until you approve" rule are all one idea: **powerful autonomy that never betrays you.** And the reliability-safe money architecture — **tools find / model reports / human submits** — is an honest answer to where LLMs actually break (the hallucination wall on long chains), instead of pretending they don't.

The arc. JARVIS vision → built it (voice, memory, desktop take-over, council, dreaming, skillsmith) → hit the real wall, which was never the *framework* (that part is solid) but **model strength on long chains + the thing needing to earn.** Hence the pivots: bug bounty ("prove $1 first, no breadth") → **AI red-teaming**, because that's the actual edge — AI × security × agent-reliability is scarce, and with the EU AI Act, demand comes *to you*. The MCP scanner and the 0din / Gray Swan path are that thesis executing.

The why underneath it. Not a portfolio toy — **leverage**: truth-machinery built from a place of having felt powerless against lies and rigged systems. Sutra as a **growth partner, not a tool** — it learns you, learns new skills, and teaches you (the pentest-tutor loop). That compounding human+AI loop *is* the product, which is why **truth-discipline** — no phantom successes, verified on disk, honest reporting — is non-negotiable inside it.

Where it honestly stands: feature-complete *in code*, validated in pieces. The remaining gap isn't building — it's **Gate-2 (real daily-driver use)** and making it **earn**.

---

1. What Sutra is

A personal "JARVIS-class" digital colleague that runs on **your own hardware**, costs **nothing by default** (local models + free tiers; paid Claude is strictly opt-in), remembers you across sessions in a **graph on your own disk**, acts on your **real desktop**, reasons as a **team of sub-agents**, **gets better the more you use it**, and can **write new skills for itself with your approval**.

North-star vision: not a tool but a *growth partner* — it learns about you, learns new skills, and **teaches you** (especially a pentest tutor as you do THM/HTB together). The compounding human+AI loop is what steers the roadmap toward a coaching surface.

The deeper "why": leverage and truth-machinery — building from a place of felt powerlessness to fight lies and rigged systems. Truth-discipline is treated as sacred.

---

2. Current status & stats (live, 2026-10-07)

Metric	Value
Package Python files	**154** (`friday/`)
Package LOC	**~30,060**
Test files / test functions	**141 files / 1,542 tests**
Git commits	**343** (first: *"week 1 foundation — router, skills, agent loop, CLI"*)
Skills	**~90** skill modules

Metric	Value
CLI commands	**40+** `sutra …` subcommands
Current branch	`main` (feature branches for red-team / MCP-scanner work)
README's own snapshot	"776 tests · 49 skills" (an earlier milestone; the project has grown well past it)

Status: feature-complete *in code* and validated in pieces; a real daily-driver personal project, not yet a polished multi-user product. Remaining work is mostly **Gate-2 live validation** and the **money pivots**, not core building.

GitHub & development process

- **Repo:** `github.com/sarjanpatel22/friday-os` — **private**, Python, no license yet, 1.6 MB.
- **Timeline:** created **2026-06-02**, active through **2026-08-23** — the bulk built in **~12 weeks**, solo.
- **Workflow:** disciplined **PR-per-feature** — **179 PRs (178 merged), 0 issues, 0 left open**. First PR 2026-06-16, last 2026-08-23. CI via GitHub Actions (caught broken clean-install + latent bugs).
- GitHub README still shows an earlier snapshot ("776 tests / 49 skills") — worth refreshing to the current **1,542 tests / ~90 skills**.
- **Not everything is pushed:** e.g. the `feat-llm-redteam-battery` branch (1,607 tests green, live-proven) is held locally pending the money push — the public/remote state can lag local.

Sarjan's wider GitHub (context, not Sutra)

`custom-ai-chatbot` + `ai-chatbot-demo` (Fiverr chatbot gig) · `web-app-security-lab`, `vulnerability-scanner`, `siem-threat-detection-lab` (3 public security-portfolio repos) · `sarjanpatel22` (profile README) · `private-workspace` (AI-kit/resume backup) · `b2b-industrial-inc` · `screenview` (oldest, pre-Sutra). **10 repos total.**

---

3. Architecture & stack

- **Router:** LiteLLM-style **tiered routing** — local Ollama → free cloud (Groq Llama 3.3 70B, Gemini Flash, NVIDIA NIM: Nemotron + GLM) → paid Claude (gated behind

`ROUTER_ALLOW_PAID=1` + a budget). Auto-route-by-effort picks the tier; prompt caching + cache-aware budget + tool-result caps keep cost near $0.

- **Council / reasoning:** Claude API (Sonnet default, Opus rare) for heavy synthesis; a **council of specialist sub-agents** fans out in parallel and synthesizes.
- **Voice:** faster-whisper Large-v3 INT8 (STT) + Kokoro (TTS) + Silero neural VAD + wake word + barge-in, ~1.2 s round-trip; Pipecat pipeline, push-to-talk default.
- **Memory:** Graphiti + **FalkorDB** graph (on your disk) + `nomic-embed` vectors in-process + a **reranked recall index** (hit@1 100% with `RECALL_RERANK`) + ambient capture. *(Qdrant was evaluated and dropped.)*
- **Vision / desktop control:** screen capture (mss ~50fps) + OpenCV template/element detect + preprocessed-region OCR (tesseract) → **real click coordinates** via xdotool (not pixel-guessing).
- **Frontend:** Tauri + Next.js + Three.js **particle orb** (gold champagne-on-obsidian + mist + voice-frequency visual). 3D graph viewer for memory.
- **Desktop:** xdotool on X11 (Kali Linux).

Hardware it was built on: RTX 3080 Ti Mobile (16 GB VRAM), 32 GB RAM, Kali Linux, X11, NVIDIA driver 550.

---

4. Module map (`friday/`)

Package	Role
`core/`	Router, agent loop, autoroute brain, council, planner, dreaming, skillsmith, coach/study/learning, doctor, monitor, stats, scope, secrets, subagents, train, persona, injection-guard, hunt/bounty, mcp-audit, redteam-report
`skills/`	~90 skills (see §5) + `registry.py` (on-demand tool selection)
`voice/`	Whisper STT, Kokoro TTS, Silero VAD, wake word, barge-in pipeline
`memory/`	Graphiti + FalkorDB graph memory, reranked recall, 3D graph viewer
`vision/`	screen capture + OCR → click coordinates (`perceive.py`)
`gui/`	particle-orb + generative-UI dashboard
`ambient/`	ambient over-the-shoulder capture/suggestions
`watch/`	`sutra watch` proactive desktop suggest-loop

Package	Role
`telegram/`	Telegram bridge (`sutra telegram`)
`rag/`	document ingest + ask (`sutra rag ingest/ask`)
`mcp/`	MCP server integration (call external tools)
`storage/`	persistence

`core/` modules (30)

`agent · autoroute · bounty · coach · commands · constants · council · doctor · dreaming · history · hunt · injection_guard · knowledge · learning · mcp_audit · monitor · persona · planner · playbook · redteam_report · router · scope · secrets · skill_profile · skillsmith · stats · study · subagents · train · turnlog`

---

5. Skills catalog (~90)

Skills are loaded **on-demand**: an always-on CORE set plus the turn-relevant specialized skills (lexical match, capped by `SUTRA_MAX_TOOLS`, default 25). This fixed the schema ceiling so raw count no longer hurts selection, and input tokens drop.

Core / everyday: `web_search · file_ops · system_info · calc · clock · weather · memory · rag · http_request (call ANY REST API) · code_exec · code_review · git_tools · dev_tools · encoding (base64/hex/url/rot13) · preferences · delegate · council · guide`

Desktop / vision: `desktop_control · desktop_input · desktop · see_screen · screen_perception · vision_agent · apps · windows · browse · browser_agent · browser_session · firefox_login · web_actions`

Recon (passive/active): `recon · recon_assets · web_recon · dns_brute · zone_transfer · crtsh · shodan_recon · github_recon · wayback · js_endpoints · param_discovery · api_discovery · waf_detect · buckets · takeover` *(+ shell-outs: dig/whois/searchsploit, nmap-free port_scan, gobuster/ffuf web_enum, nikto, whatweb, sslscan, hydra, nuclei)*

Web-vuln testing: `sqli · nosqli · xxe · ssrf · ssti · csrf · cors · crlf · open_redirect · path_traversal · host_header · dom_xss · reflected_xss · access_control · cache_poison · http_methods · graphql · graphql_batch · email_security · csp_analyze · secret_scan · injection_scan`

AI / LLM security (the current money edge): `llm_redteam · agentic_redteam · agentic_target · prompt_injection · mcp_audit`

Loot / reversing: `reversing · disassemble` + hash family (identify/hash/crack), `code_exec`

Infra / ops: `connect_vpn` (scoped NOPASSWD openvpn), `auth`, `access_control`

---

6. CLI command reference (`sutra …`)

Core: `start` (boots orb+chat) · `chat` · `ask` (one-shot) · `think` · `skills` · `devices` · `status` · `doctor` (preflight: what's installed/missing + how to fix) · `budget` · `logs` · `stats`

Voice & GUI: `voice` (PTT/wake) · `orb` · `dashboard` · `graph` (3D memory viewer) · `ambient`

Desktop acting: `do` (act on screen) · `guide` (show-me-the-button) · `watch` (proactive loop)

Memory / knowledge: `rag ingest` · `rag ask`

Reasoning / agents: `delegate` · `subagents` · `council` · `monitor`

Planning: `plan run|do|resume|list|status` (decompose a goal → small steps → save after each → resume exactly where it stopped)

Autonomous security (gated, authorized use only): `auto` · `pwn` · `bounty` · `hunt` · `redteam` · `agentic-redteam` · `agentic-target` · `scan-injection` · `mcp-audit` · `connect_vpn/disconnect_vpn/vpn_status`

Learning loop: `train` · `study` · `learn` · `coach` · `learning` · `tutor`

Self-extension: `skill draft|list|show|approve|reject` (drafts a new skill → **you approve** → it loads; inert source until approved)

Integrations: `mcp` · `secrets` · `telegram`

---

7. The build journey (phases & pivots)

Week 1 — Foundation. Router, skills registry, agent loop, CLI. All tests green from day one.

JARVIS vision built (mid-June 2026). Substantive chat via free 70B (not tiny models), `sutra watch` proactive suggest-loop, `guide_me` show-me-the-button, screen take-over (`act_on_screen` / `type_into` via OCR→xdotool with real coords). Recall reranker → hit@1 100%. From-zero gap audit (fixed paid-tier leak, voice gate bypass, MCP double-init race, Qdrant removal).

v1 phases shipped one-by-one. Wake word, Telegram, observability (`sutra logs`), generative-UI dashboard, orb. → **v1 feature-complete in code; remaining = Gate-2 live validation.**

Autonomous-pentest stack. Sub-agent council (`convene`), scoped VPN, `sutra auto` / `pwn` loop with **hybrid-confirm** hardening (auto-runs clean recon, asks before risky actions), `is_catastrophic` floor, engagement **scope guard**, free-worker **$0 guarantee** under fan-out. *Refused to build a reCAPTCHA solver (ToS) — and keeps refusing.*

Skills scaling. On-demand tool loading (the ceiling fix), self-extending `create_skill`, pentest skill batches: encode/decode + hashes → recon (dns/whois/searchsploit/port_scan) → web-recon (gobuster/ffuf, http_headers) → loot (crack_hash via John).

Task planner (SHIPPED). `friday/core/planner.py` + `sutra plan` — the "divide the work" fix for the core frustration (can't last 3 min / can't divide / rate-limits). Strong model decomposes once → execute one step at a time → SAVE after each → pause on budget/error → `resume`. Stress-tested: free/local did 20/22 varied tasks in 3.5 min, $0, zero phantom successes.

Cost stack (PAID Claude wired, opt-in). Prompt caching (96% prefix cached @10%), auto-route-by-effort, cache-aware budget, tool-result cap. Key finding: *the framework is flawless; the only ceiling is model strength on long chains.*

Router brains. GLM + Nemotron as free NVIDIA NIM tiers (GLM leads REASON/HEAVY, Nemotron MEDIUM). Later: GLM-5.1→5.2 bump after EOL, with a `doctor` liveness check.

Graph session-capture. A SessionEnd hook feeds every Claude Code session into the FalkorDB graph brain (detached ingest so exit never blocks).

Fast screen perception. mss ~50fps + OpenCV detect + region OCR — fixed flaky desktop take-over. Live win: Sutra auto-downloaded a THM VPN config via a scroll-and-detect loop.

The money pivots

1. → **Bug bounty (2026-06-24).** Make Sutra earn. Narrow-first; reliability-safe architecture = **TOOLS find / MODEL reports / HUMAN submits** (dodges the hallucination wall). "Prove $1 first."
2. → **AI red-teaming (2026-07-08).** The real edge = AI × security × agent-reliability (scarce, demand-comes-to-you, EU-AI-Act catalyst). Built `redteam_llm` battery + `sutra redteam` CLI + report drafter; black-box judge mode; agentic prompt-injection battery (0din Read/Write/Execute violations). Money map: 0din.ai / Gray Swan Arena / OpenAI bounty.
3. **MCP security scanner (latest, #174–#179).** Hardened the agent loop against indirect prompt injection (LLM01), standalone prompt-injection detector (scan API/skill/CLI/benchmark), and an **MCP vulnerability scanner** (reachability, guard-bypass, SQLi detector, multi-hop taint).

---

8. Key design principles

- **$0 by default** — every call prefers local/free; paid is flag- + budget-gated.
- **Confirmation gate** — nothing irreversible runs without approval; enforced *and audited in CI*. `is_catastrophic` blocks home/dotdir wipes; `_RISKY_CMD` catches persistence/wipe patterns.
- **Self-extension is safe** — drafted skills are inert source until *you* approve them.
- **Reliability-safe for money** — tools find, the model reports, the human submits.
- **Scope-restricted autonomy** — `SUTRA_SCOPE` / `--scope`; off-scope targets escalate to operator.
- **Local-first sovereignty** — memory graph + data never leave the machine.
- **Truth-discipline** — no phantom successes; verified on disk; honest reporting.

---

9. Notable subsystems

- **Planner** — decompose → step-by-step → save/resume (`sutra plan`).
- **Dreaming** — records what worked/failed, recalls it on similar tasks (learn from experience).
- **Council / sub-agents** — parallel specialist fan-out + synthesis; free-worker $0 guarantee.
- **Skillsmith** — drafts new skills from a description for your approval.
- **Coach / tutor / study / learn** — mentor mode that explains the *why* and gives the next step.
- **Ambient / watch** — over-the-shoulder proactive suggestions.
- **Graph memory + reranked recall** — durable, searchable, on-disk; 3D viewer.

- **Doctor** — preflight that lists every dep/binary/skill/service + exact fix command.

---

10. Hardening & hygiene

Full 159-file audit: architecture SOLID; the gap was project hygiene. Shipped: core deps to base + resilient autodiscover, `sutra doctor`, CI (caught broken clean-install + latent bugs). Class to watch: tests/skills secretly depending on local env (DISPLAY/binaries/.env keys/optional deps). Known debt: `load_dotenv`-at-import smell in the router (lazy-config vs entrypoint ordering).

---

11. Documentation index (`docs/`)

Doc	What
`SUTRA_PHASES.md`	**Authoritative phase/state doc** (verify state here + git log)
`GAP_AUDIT_2026-06-15.md`	From-zero gap audit + deferred gaps
`SKILLS_ROADMAP.md`	Scale via MCP + on-demand loading + self-extending skills
`GUI_ORB_HANDOFF.md` / `FRIDAY_VISUAL_DIRECTION.md`	Orb/visual direction
`MODEL_REFRESH_2026-06.md`	Model/router refresh notes
`LIVE_TEST_CHECKLIST.md`	Gate-2 live-validation checklist
`MEMORY_VIEWER_AND_AMBIENT.md`	Memory viewer + ambient design
`FRIDAY_FINAL_SPEC.md` / `FRIDAY_HANDOVER.md` / `FRIDAY_REALITY_CHECK.md`	Original spec/handover
`ROADMAP_BRAINSTORM_2026.md` / `MODERNIZATION_AUDIT.md`	Roadmap + modernization

Doc	What
`CLAUDE.md` (root)	Project conventions for AI pair-programming

Note: `SESSION_HANDOFF.md` is stale — trust `SUTRA_PHASES.md` + git log + the gap audit.

---

12. What's next

- **Gate-2:** Sarjan's own live validation + daily-driver use (the main remaining "work").
- **Money:** push/submit the red-team work — black-box judge mode → 0din / Gray Swan / OpenAI bounty. (Branch `feat-llm-redteam-battery`: 1607 tests green, live-proven vs local Ollama, **not pushed**.)
- **Skills:** trim unused (per-skill usage logs populate on next live run) before adding more; next offensive skills (heavier ones) only with explicit go.
- **Icon-only controls:** vision currently finds text; icon-only buttons are the next perception gap.

This overview is a snapshot. Ground truth = the code + `docs/SUTRA_PHASES.md` + `git log`.